D Y PATIL UNIVERSITY
NAVI MUMBAI

Subject Name : Python Programming

Unit No: 2

Unit Name : Functions and Modules

Faculty Name : Prachi Abhijeet Surve

D Y PATIL
UNIVERSITY
NAVI MUMBAI

Unit No.: 2

Faculty Name :

Index

Lecture No & Topic Name	Slide No
Functions and Modules	
Built-in Functions	
User Defined Functions	
Importing modules	
DateTime Module	
Math Module	
String Module	
Random Module	

D Y PATIL UNIVERSITY
NAVI MUMBAI

Lecture No: 1

Python Functions

A function is a block of code which only runs when it is called.

A function can return data as a result.

A function helps avoiding code repetition.

Creating a Function

In Python, a function is defined using the `def` keyword, followed by a function name and parentheses:

Example

```
def my_function():
  print("Hello from a function")
```

This creates a function named `my_function` that prints "Hello from a function" when called.

The code inside the function must be indented.

Python uses indentation to define code blocks.

```
def my_function():
  print("Hello from a function")

my_function()
```

```
Hello from a function
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Definition

A **function** is a block of reusable code that performs a specific task.
Functions help in **modularity, code reuse, and readability**.

Python provides:

- **Built-in functions** (e.g., `print()`, `len()`, `sum()`)
- **User-defined functions** (created using `def`)

Syntax of a Function

```
def function_name(parameters):
    statement(s)
    return value
```

Example:

```
def greet():
    print("Hello, World!")

greet()
```

```
def my_function():
  print("Hello from a function")

my_function()
my_function()
my_function()
```

```
Hello from a function
Hello from a function
Hello from a function
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Types of Functions Based on Arguments and Return Value:

Type	Description	Example
No arguments, no return	Performs task only	`def show(): print("Hi")`
Arguments, no return	Takes input, no output	`def show(a): print(a)`
No arguments, return value	Returns result	`def get(): return 5`
Arguments and return value	Most common type	`def add(a,b): return a+b`

D Y PATIL UNIVERSITY
NAVI MUMBAI

How to Use Built-in Function in Python?

To use **built-in functions** in your code, simply call the specific function by passing the required parameter (if any) inside the parentheses. Since these functions are pre-defined, you don't need to import any module or package.

Example of Using Built-in Functions

Consider the following example demonstrating the use of built-in functions in your code:

```
# Using print() and len() function

text = "Tutorials Point"

print(len(text)) # Prints 15
```

In the above example, we are using two built-in functions print() and len().

D Y PATIL UNIVERSITY
NAVI MUMBAI

Function	Meaning	Example	Output
abs()	Returns absolute (positive) value	abs(-10)	10
all()	True if all items are true	all([True,True,False])	False
any()	True if any item is true	any([False,True])	True
ascii()	Readable string with escaped non-ascii	ascii('Swati©')	'Swati\xa9'
bin()	Converts number to binary	bin(10)	'0b1010'
bool()	Returns boolean value	bool(0)	False
bytearray()	Returns mutable byte array	bytearray(4)	bytearray(...)
bytes()	Returns immutable bytes object	bytes(4)	b'\x00...'
callable()	Checks if object is callable	callable(len)	True
chr()	Returns character from Unicode	chr(65)	'A'
complex()	Creates complex number	complex(2,3)	(2+3j)
dict()	Creates dictionary	dict(name='Swati')	{'name':'Swati'}
dir()	Lists attributes of object	dir(str)	List of methods
divmod()	Returns quotient & remainder	divmod(10,3)	(3,1)

enumerate()	Adds index to iterable	list(enumerate(['A','B']))	[(0,'A'),(1,'B')]
eval()	Evaluates expression	eval('5+3')	8
exec()	Executes Python code	exec('print(5)')	5
filter()	Filters items by condition	list(filter(lambda x:x>2,[1,2,3]))	[3]
float()	Converts to float	float(5)	5.0
format()	Formats value	format(255,'b')	'11111111'
frozenset()	Creates immutable set	frozenset([1,2,2])	frozenset({1,2})
getattr()	Gets attribute value	getattr(str,'upper')	Method ref
hasattr()	Checks attribute exists	hasattr(str,'lower')	True
hash()	Returns hash value	hash('Python')	Integer
hex()	Converts to hexadecimal	hex(255)	'0xff'
id()	Returns object identity	id(10)	Memory address
int()	Converts to integer	int('10')	10
isinstance()	Checks object type	isinstance(5,int)	True
issubclass()	Checks subclass relation	issubclass(bool,int)	True

len()	Returns length of object	len('Python')	6
list()	Creates list	list((1,2,3))	[1,2,3]
map()	Applies function to items	list(map(lambda x:x*2,[1,2]))	[2,4]
max()	Returns largest value	max([1,5,3])	5
min()	Returns smallest value	min([1,5,3])	1
next()	Returns next iterator item	next(iter([1,2]))	1
oct()	Converts to octal	oct(10)	'0o12'
ord()	Returns Unicode of character	ord('A')	65
pow()	Returns power value	pow(2,3)	8
print()	Prints output	print('Hello')	Hello
range()	Generates sequence of numbers	list(range(3))	[0,1,2]
repr()	Returns official string form	repr('Hi')	'Hi'
reversed()	Returns reverse iterator	list(reversed([1,2,3]))	[3,2,1]

round()	Rounds number	round(3.567,2)	3.57
set()	Creates set	set([1,1,2])	{1,2}
slice()	Creates slice object	a=[1,2,3]; a[slice(0,2)]	[1,2]
sorted()	Returns sorted list	sorted([3,1,2])	[1,2,3]
str()	Converts to string	str(100)	'100'
sum()	Adds iterable items	sum([1,2,3])	6
tuple()	Creates tuple	tuple([1,2])	(1,2)
type()	Returns object type	type(10)	<class 'int'>
vars()	Returns object's __dict__	vars(obj)	Dictionary
zip()	Combines iterables	list(zip([1,2],['A','B']))	[(1,'A'),(2,'B')]

D Y PATIL UNIVERSITY
NAVI MUMBAI

Built-in Functions

Python provides a lot of built-in functions that ease the writing of code.

Python's built-in functions, exploring their various applications and highlighting some of the most commonly used ones.

Built-in Functions

A
abs()
aiter()
all()
anext()
any()
ascii()

B
bin()
bool()
breakpoint()
bytearray()
bytes()

C
callable()
chr()
classmethod()
compile()
complex()

D
delattr()
dict()
dir()
divmod()

E
enumerate()
eval()
exec()

F
filter()
float()
format()
frozenset()

G
getattr()
globals()

H
hasattr()
hash()
help()
hex()

I
id()
input()
int()
isinstance()
issubclass()
iter()

L
len()
list()
locals()

M
map()
max()
memoryview()
min()

N
next()

O
object()
oct()
open()
ord()

P
pow()
print()
property()

R
range()
repr()
reversed()
round()

S
set()
setattr()
slice()
sorted()
staticmethod()
str()
sum()
super()

T
tuple()
type()

V
vars()

Z
zip()

_
__import__()

Advantages of Using Built-in Functions

The following are the advantages of using built-in functions:

- The use of the built-in functions simplifies and reduces the code length and enhances the readability of the code.
- Instead of writing the same logic repeatedly, you can use these functions across different sections of the program.
- This not only saves time but also helps in maintaining consistency of code.
- These functions provide a wide range of functionalities including mathematical operations, data type conversion, and performing operations on iterators.
- These functions have descriptive names that make the code easier to understand and maintain.
- Developers need not write additional complex code for performing certain operations.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Parameters vs Arguments

- The terms *parameter* and *argument* can be used for the same thing: information that are passed into a function.

From a function's perspective:

- A parameter is the variable listed inside the parentheses in the function definition.
- An argument is the actual value that is sent to the function when it is called.

Example

```
def my_function(name): # name is a parameter
  print("Hello", name)

my_function("Emil") # "Emil" is an argument
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Arguments

- Information can be passed into functions as arguments.
- Arguments are specified after the function name, inside the parentheses.
- You can add as many arguments as you want, just separate them with a comma.
- The following example has a function with one argument (fname).
- When the function is called, we pass along a first name, which is used inside the function to print the full name:

```
def my_function(fname):
  print(fname + " Refsnes")

my_function("Emil")
my_function("Tobias")
my_function("Linus")
```

```
Emil Refsnes
Tobias Refsnes
Linus Refsnes
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Number of Arguments

By default, a function must be called with the correct number of arguments.

If your function expects 2 arguments, you must call it with exactly 2 arguments.

```python
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil", "Refsnes")
```

```
Emil Refsnes
```

```python
def my_function(fname, lname):
  print(fname + " " + lname)

my_function("Emil")
```

```
Traceback (most recent call last):
File "demo_function_arg_error.py", line 4, in <module>
my_function("Emil")
TypeError: my_function() missing 1 required positional argument: 'lname'
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Default Parameter Values

You can assign default values to parameters.

If the function is called without an argument, it uses the default value:

```python
def my_function(name = "friend"):
  print("Hello", name)

my_function("Emil")
my_function("Tobias")
my_function()
my_function("Linus")
```

```
Hello Emil
Hello Tobias
Hello friend
Hello Linus
```

```python
def my_function(country = "Norway"):
  print("I am from", country)

my_function("Sweden")
my_function("India")
my_function()
my_function("Brazil")
```

```
I am from Sweden
I am from India
I am from Norway
I am from Brazil
```

Keyword Arguments

You can send arguments with the key = value syntax.

```python
def my_function(animal, name):
  print("I have a", animal)
  print("My", animal + "'s name is", name)

my_function(animal = "dog", name = "Buddy")
```

```
I have a dog
My dog's name is Buddy
```

This way, with keyword arguments, the order of the arguments does not matter.

```python
def my_function(animal, name):
  print("I have a", animal)
  print("My", animal + "'s name is", name)

my_function(name = "Buddy", animal = "dog")
```

```
I have a dog
My dog's name is Buddy
```

The phrase *Keyword Arguments* is often shortened to *kwargs* in Python documentation.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Positional Arguments

When you call a function with arguments without using keywords, they are called positional arguments.

Positional arguments must be in the correct order:

```
def my_function(animal, name):
  print("I have a", animal)
  print("My", animal + "'s name is", name)

my_function("dog", "Buddy")
```

```
I have a dog
My dog's name is Buddy
```

The order matters with positional arguments:

```
def my_function(animal, name):
  print("I have a", animal)
  print("My", animal + "'s name is", name)

my_function("Buddy", "dog")
```

```
I have a Buddy
My Buddy's name is dog
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Mixing Positional and Keyword Arguments

You can mix positional and keyword arguments in a function call.
However, positional arguments must come before keyword arguments:

```
def my_function(animal, name, age):
  print("I have a", age, "year old", animal, "named", name)

my_function("dog", name = "Buddy", age = 5)
```

```
I have a 5 year old dog named Buddy
```

Passing Different Data Types

You can send any data type as an argument to a function (string, number, list, dictionary, etc.).
The data type will be preserved inside the function:

```
def my_function(fruits):
  for fruit in fruits:
    print(fruit)

my_fruits = ["apple", "banana", "cherry"]
my_function(my_fruits)
```

```
apple
banana
cherry
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

```python
def my_function(person):
  print("Name:", person["name"])
  print("Age:", person["age"])

my_person = {"name": "Emil", "age": 25}
my_function(my_person)
```

```
Name: Emil
Age: 25
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Passing Different Data Types

You can send any data type as an argument to a function (string, number, list, dictionary, etc.).

The data type will be preserved inside the function:

```
def my_function(fruits):
  for fruit in fruits:
    print(fruit)

my_fruits = ["apple", "banana", "cherry"]
my_function(my_fruits)
```

```
apple
banana
cherry
```

```
def my_function(person):
  print("Name:", person["name"])
  print("Age:", person["age"])

my_person = {"name": "Emil", "age": 25}
my_function(my_person)
```

```
Name: Emil
Age: 25
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Return Values

Functions can return values using the return statement:

```python
def my_function(x, y):
  return x + y

result = my_function(5, 3)
print(result)
```

```
8
```

```python
def my_function():
  return (10, 20)

x, y = my_function()
print("x:", x)
print("y:", y)
```

```
x: 10
y: 20
```

Returning Different Data Types

Functions can return any data type, including lists, tuples, dictionaries, and more.

```python
def my_function():
  return ["apple", "banana", "cherry"]

fruits = my_function()
print(fruits[0])
print(fruits[1])
print(fruits[2])
```

```
apple
banana
cherry
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Positional-Only Arguments

You can specify that a function can have ONLY positional arguments.
To specify positional-only arguments, add , / after the arguments:

```python
def my_function(name, /):
  print("Hello", name)

my_function("Prachi")
```

```
Hello Prachi
```

Without the `, /` you are actually allowed to use keyword arguments even if the function expects positional arguments:

```python
def my_function(name):
  print("Hello", name)

my_function(name = "Emil")
```

```
Hello Emil
```

With `, /`, you will get an error if you try to use keyword arguments:

```python
def my_function(name, /):
  print("Hello", name)

#This will cause an error:
my_function(name = "Emil")
```

```
Traceback (most recent call last):
  File "./prog.py", line 5, in <module>
TypeError: my_function() got some positional-only arguments passed as keyword arguments: 'name'
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Keyword-Only Arguments

To specify that a function can have only keyword arguments, add *, before the arguments:

```python
def my_function(*, name):
  print("Hello", name)

my_function(name = "Emil")
```

```
Hello Emil
```

Without `*,` , you are allowed to use positional arguments even if the function expects keyword arguments:

```python
def my_function(name):
  print("Hello", name)

my_function("Emil")
```

```
Hello Emil
```

With `*,` , you will get an error if you try to use positional arguments:

```python
def my_function(*, name):
  print("Hello", name)

my_function("Emil")
```

```
Traceback (most recent call last):
  File "./prog.py", line 4, in <module>
TypeError: my_function() takes 0 positional arguments but 1 was given
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Combining Positional-Only and Keyword-Only

You can combine both argument types in the same function.
Arguments before / are positional-only, and
arguments after * are keyword-only:

```
def my_function(a, b, /, *, c, d):
  return a + b + c + d

result = my_function(5, 10, c = 15, d = 20)
print(result)
```

```
50
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

*args and **kwargs

- By default, a function must be called with the correct number of arguments.
- However, sometimes you may not know how many arguments that will be passed into your function.
- *args and **kwargs allow functions to accept a unknown number of arguments.

Arbitrary Arguments - *args

- If you do not know how many arguments will be passed into your function, add a * before the parameter name.
- This way, the function will receive a *tuple* of arguments and can access the items accordingly:

```
def my_function(*kids):
  print("The youngest child is " + kids[2])

my_function("Emil", "Tobias", "Linus")
```

```
The youngest child is Linus
```

Arbitrary Arguments are often shortened to **args* in Python documentation.

D Y PATIL UNIVERSITY NAVI MUMBAI

What is *args?

The *args parameter allows a function to accept any number of positional arguments.

Inside the function, args becomes a tuple containing all the passed arguments:

```python
def my_function(*args):
  print("Type:", type(args))
  print("First argument:", args[0])
  print("Second argument:", args[1])
  print("All arguments:", args)

my_function("Emil", "Tobias", "Linus")
```

```
Type: <class 'tuple'>
First argument: Emil
Second argument: Tobias
All arguments: ('Emil', 'Tobias', 'Linus')
```

Practical Example with *args

*args is useful when you want to create flexible functions:

Example

A function that calculates the sum of any number of values:

```python
def my_function(*numbers):
  total = 0
  for num in numbers:
    total += num
  return total

print(my_function(1, 2, 3))
print(my_function(10, 20, 30, 40))
print(my_function(5))
```

```
6
100
5
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

```python
def my_function(*numbers):
  if len(numbers) == 0:
    return None
  max_num = numbers[0]
  for num in numbers:
    if num > max_num:
      max_num = num
  return max_num

print(my_function(3, 7, 2, 9, 1))
```

```
9
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Arbitrary Keyword Arguments - **kwargs

If you do not know how many keyword arguments will be passed into your function, add two asterisks ** before the parameter name.

This way, the function will receive a *dictionary* of arguments and can access the items accordingly:

Example

Using **kwargs to accept any number of keyword arguments:

```
def my_function(**kid):
  print("His last name is " + kid["lname"])

my_function(fname = "Tobias", lname = "Refsnes")
```

```
His last name is Refsnes
```

Arbitrary Keyword Arguments are often shortened to ***kwargs* in Python documentation.

D Y PATIL UNIVERSITY
NAVI MUMBAI

What is **kwargs?

The `**kwargs` parameter allows a function to accept any number of keyword arguments.

Inside the function, `kwargs` becomes a dictionary containing all the keyword arguments:

Example

Accessing values from **kwargs:

```
def my_function(**myvar):
  print("Type:", type(myvar))
  print("Name:", myvar["name"])
  print("Age:", myvar["age"])
  print("All data:", myvar)

my_function(name = "Tobias", age = 30, city = "Bergen")
```

```
Type: <clas
Name: Tobia
Age: 30
All data: {
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Using **kwargs with Regular Arguments

You can combine regular parameters with **kwargs.

Regular parameters must come before **kwargs:

```
def my_function(username, **details):
  print("Username:", username)
  print("Additional details:")
  for key, value in details.items():
    print("  ", key + ":", value)

my_function("emil123", age = 25, city = "Oslo", hobby = "coding")
```

```
Username: emil123
Additional details:
age: 25
city: Oslo
hobby: coding
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Combining *args and **kwargs

You can use both *args and **kwargs in the same function.

The order must be:

1. regular parameters
2. *args
3. **kwargs

```
Title: User Info
Positional arguments: ('Emil', 'Tobias')
Keyword arguments: {'age': 25, 'city': 'Oslo'}
```

```python
def my_function(title, *args, **kwargs):
  print("Title:", title)
  print("Positional arguments:", args)
  print("Keyword arguments:", kwargs)

my_function("User Info", "Emil", "Tobias", age = 25, city = "Oslo")
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Unpacking Arguments

The * and ** operators can also be used when *calling* functions to unpack (expand) a list or dictionary into separate arguments.

Unpacking Lists with *

If you have values stored in a list, you can use * to unpack them into individual arguments:

Example

Using * to unpack a list into arguments:

```
def my_function(a, b, c):
  return a + b + c

numbers = [1, 2, 3]
result = my_function(*numbers) # Same as: my_function(1, 2, 3)
print(result)
```

```
6
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Unpacking Dictionaries with **

If you have keyword arguments stored in a dictionary, you can use ** to unpack them:

Example

Using ** to unpack a dictionary into keyword arguments:

```
def my_function(fname, lname):
  print("Hello", fname, lname)

person = {"fname": "Emil", "lname": "Refsnes"}
my_function(**person) # Same as: my_function(fname="Emil", lname="Refsnes")
```

```
Hello Emil Refsnes
```

Remember: Use * and ** in function definitions to *collect* arguments, and use them in function calls to *unpack* arguments.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Recursive Functions

A function that calls itself.

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n-1)

print(factorial(5))   # 120
```

Lambda Functions (Anonymous Functions)

- Small, one-line functions.
- Defined using `lambda` keyword.

Syntax:

```
lambda arguments: expression
```

Example:

```
square = lambda x: x*x
print(square(4))   # 16
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Unit No:2

Lecture No: 2
Module

Python Modules

A Module in python is a file containing definitions and statements.

A module can define functions, classes and variables.

Modules help organize code into separate files so that programs become easier to maintain and reuse.

Instead of writing everything in one place, related functionality can be grouped into its own module and imported whenever needed.

Import module

Modules can be used in another Python file using the import statement. When Python sees an import, it loads the module if it exists in the interpreter's search path. Below is the syntax to import a module:

import module

Example: Here, we are importing the **calc** that we created earlier to perform add operation.

```
import calc
print(calc.add(10, 2))
```

Output

```
12
```

Explanation: import calc loads the module and calc.add() accesses a function through dot notation.

UNIVERSITY
NAVI MUMBAI

Types of Import Statements:

1. Import From Module: This allows importing specific functions, classes, or variables rather than the whole module.

```
from math import sqrt, factorial
print(sqrt(16))
print(factorial(6))
```

Output

```
4.0
720
```

Explanation: Only sqrt and factorial are brought into the local namespace, so the prefix math. is not required.

2. Import All Names: * imports everything from a module into the current namespace.

```
from math import *
print(sqrt(16))
print(factorial(6))
```

Output

```
4.0
720
```

3. Import With Alias: You can shorten a module's name using as.

```
import math as m
print(m.pi)
```

Output

```
3.141592653589793
```

Explanation: math is accessed through the shorter alias m.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Locating a Module

Python searches for modules in a predefined list of directories known as the module search path. You can view this list using **sys.path**.

```
import sys
for p in sys.path:
    print(p)
```

Output

```
/home/guest/sandbox
/usr/local/lib/python313.zip
/usr/local/lib/python3.13
/usr/local/lib/python3.13/lib-dynload
/usr/local/lib/python3.13/site-packages
```

Explanation: Python checks each path in order until it finds the module you're trying to import.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Types of Modules : Python provides several kinds of modules. Each type plays a different role in application development.

1. Built-in Modules: These come bundled with Python and require no installation - **e.g.**, math, random, os.

```
import random
print(random.randint(1, 5))
```

Output

```
4
```

Explanation: random.randint() returns a random number within the given range.

D Y PATIL UNIVERSITY
NAVI MUMBAI

2. User-Defined Modules: These are modules you create yourself, such as **calc.py**.

```
import calc
print(calc.sub(20, 5))
```

Output

```
15
```

Explanation: The module is created manually and then imported into another script.

D Y PATIL UNIVERSITY
NAVI MUMBAI

3. External (Third-Party) Modules: These modules are installed using pip - **e.g.**, NumPy, Pandas, Requests.

```
import requests
r = requests.get("https://example.com")
print(r.status_code)
```

Output

```
200
```

Explanation: requests is installed separately (pip install requests) and provides HTTP utilities.

D Y PATIL UNIVERSITY
NAVI MUMBAI

4. Package Modules: A package is a directory containing multiple modules, usually with an **__init__.py** file.

Example Directory

```
mypkg/
    __init__.py
    calc.py
    utils.py
```

Using a module from a package

```
from mypkg import utils
print(utils.some_func())
```

calls a function named some_func(), the output will be whatever that function returns.

If utils.py contains something like:

```
def some_func():
    return "Hello"
```

Output

```
Hello
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Python datetime module

In Python, **date** and **time** are not built-in types but are handled using built-in **datetime** module. This module offers classes to efficiently work with dates, times and intervals, providing many useful methods. Date and DateTime are objects, so manipulating them means working with objects, not plain strings or timestamps.

Why do we need Datetime module?

1. Helps work with dates and times in real-world applications like scheduling or logging.
2. Allows easy calculation of differences between two dates or times.
3. Supports formatting and parsing of date/time strings for user-friendly outputs.
4. Useful for time-stamping events, files or data entries.
5. Essential for handling time zones, durations and calendar-based operations.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Commonly used classes in datetime module

Class	Description
date	Represents a date (year, month, day) in the Gregorian calendar
time	Represents a time independent of any date (hour, minute, second, microsecond, tzinfo)
datetime	Combines date and time (year, month, day, hour, minute, second, microsecond, tzinfo)
timedelta	Represents difference between two dates or times
tzinfo	Abstract base class for timezone information
timezone	Fixed offset timezone class (subclass of tzinfo) introduced in Python 3.2

D Y PATIL UNIVERSITY
NAVI MUMBAI

Date class

The date class provided by datetime module, is used to create and manipulate date objects. When an instance of this class is created, it represents a specific calendar date in ISO 8601 format: **YYYY-MM-DD**.

Syntax

class datetime.date(year, month, day)

Parameters:

- **year:** An integer between MINYEAR (usually 1) and MAXYEAR (usually 9999).
- **month:** An integer from 1 (January) to 12 (December).
- **day:** An integer valid for the specified month and year (e.g., 28 or 29 for February, depending on leap year).

Important Notes

- *Providing invalid types (e.g., string instead of int) raises a TypeError.*
- *Providing out-of-range values raises a ValueError.*
- *The date object does not include time or timezone information — for that, use datetime.depending on leap year).*

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example 1: Creating a Date Object

```
from datetime import date
d = date(1996, 12, 11)
print(d)
```

Output

```
1996-12-11
```

Example 2: Get Current Date

```
from datetime import date
t = date.today()
print(t)
```

Output

```
2025-07-26
```

Example 3: Accessing Year, Month and Day Attributes

```
from datetime import date
t = date.today()
print(t.year)
print(t.month)
print(t.day)
```

Output

```
2025
7
26
```

Example 4: Create Date from Timestamp

```
from datetime import datetime
date_time = datetime.fromtimestamp(1887639468)
print(date_time)
print(date_time.date())
```

Output

```
2029-10-25 16:17:48
2029-10-25
```

Example 5: Convert Date to String

```
from datetime import date
t = date.today()
date_str = t.isoformat()
print(date_str)
print(type(date_str))
```

Output

```
2025-07-26
<class 'str'>
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Important Date Class Methods

Function Name Description

- **ctime()** Return a string representing the date
- **fromisocalendar()** Returns a date corresponding to the ISO calendar
- **fromisoformat()** Returns a date object from the string representation of the date
- **fromordinal()** Returns a date object from the proleptic Gregorian ordinal, where January 1 of year 1 has ordinal 1
- **fromtimestamp()** Returns a date object from the POSIX timestamp
- **isocalendar()** Returns a tuple year, week, and weekday
- **isoformat()** Returns the string representation of the date
- **isoweekday()** Returns the day of the week as an integer where Monday is 1 and Sunday is 7
- **replace()** Changes the value of the date object with the given parameter
- **strftime()** Returns a string representation of the date with the given format
- **timetuple()** Returns an object of type time.struct_time
- **today()** Returns the current local date
- **toordinal()** Return the proleptic Gregorian ordinal of the date, where January 1 of year 1 has ordinal 1
- **weekday()** Returns the day of the week as integer where Monday is 0 and Sunday is 6

D Y PATIL UNIVERSITY
NAVI MUMBAI

Time class

The time class in Python represents a specific time of day, independent of any particular date. It is used to create time objects that include hour, minute, second, microsecond and timezone information.

Syntax

```
class datetime.time(hour=0, minute=0, second=0, microsecond=0, tzinfo=None)
```

All the arguments are optional. tzinfo can be None otherwise all the attributes must be integer in following range -

- 0 <= hour < 24
- 0 <= minute < 60
- 0 <= second < 60
- 0 <= microsecond < 1000000

D Y PATIL UNIVERSITY
NAVI MUMBAI

Example 1: Time object representing time in Python

```python
from datetime import time

# Create time object with hour, minute and second
my_time = time(13, 24, 56)
print("Entered time:", my_time)

# Time object with only minute specified
my_time = time(minute=12)
print("Time with one argument:", my_time)

# Time object with default (00:00:00)
my_time = time()
print("Time without argument:", my_time)

# time(hour=26)      → ValueError: hour must be in 0..23
# time(hour='23')    → TypeError: string passed instead of int
```

Output

```
Entered time: 13:24:56
Time with one argument: 00:12:00
Time without argument: 00:00:00
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

List of Time class Methods

Function Name	Description
dst()	Returns tzinfo.dst() is tzinfo is not None
fromisoformat()	Returns a time object from the string representation of the time
isoformat()	Returns the string representation of time from the time object
replace()	Changes the value of the time object with the given parameter
strftime()	Returns a string representation of the time with the given format
tzname()	Returns tzinfo.tzname() is tzinfo is not None
utcoffset()	Returns tzinfo.utcffsets() is tzinfo is not None

D Y PATIL UNIVERSITY
NAVI MUMBAI

1. Core Classes

The module consists of four primary classes you'll use most frequently:

- **date**: Manipulates dates (Year, Month, Day) assuming the Gregorian calendar.
- **time**: Manipulates time (Hour, Minute, Second, Microsecond) independent of any day.
- **datetime**: A combination of date and time.
- **timedelta**: Represents the **duration** or difference between two dates or times.

DateTime module is used to **handle date and time operations**.

Important classes: date, time, datetime, timedelta.

strftime() → date to string.

strptime() → string to date.

timedelta → used for date arithmetic.

D Y PATIL UNIVERSITY
NAVI MUMBAI

Working with `datetime` Objects

To use the module, you must first import it. The most common entry point is `datetime.datetime`.

Getting Current Date and Time

Python

```
from datetime import datetime

# Get current local date and time
now = datetime.now()
print(f"Current Timestamp: {now}")

# Get specific components
print(f"Year: {now.year}, Month: {now.month}, Day: {now.day}")
```

Creating Specific Instances

Python

```
# datetime(year, month, day, hour, minute, second)
dt = datetime(2025, 12, 25, 10, 30)
print(dt) # 2025-12-25 10:30:00
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Formatting

Code	Meaning	Example
%a	Abbreviated weekday	Sun
%A	Full weekday	Sunday
%b	Abbreviated month	Sep
%B	Full month	September
%p	AM/PM	PM
%d	Day	18
%m	Month	02
%Y	Year (4 digits)	2026
%y	Year (2 digits)	26
%H	Hour (24-hour)	14
%I	Hour (12-hour)	02
%M	Minutes	30
%S	Seconds	45

D Y PATIL UNIVERSITY NAVI MUMBAI

Important Functions Summary

Function	Purpose
date.today()	Get current date
datetime.now()	Get current date and time
strftime()	Format date/time as string
strptime()	Convert string to date/time
timedelta()	Add or subtract time

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Math Module

Introduction

The **Math module** in Python provides **mathematical functions** to perform operations such as:

- Trigonometric calculations
- Logarithms
- Powers and roots
- Rounding and absolute values
- Constants like π and e

The math module is part of Python's **standard library**.

```
import math
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Power and Root Functions

Function	Description	Example
math.sqrt(x)	Square root of x	sqrt(25) = 5
math.pow(x, y)	x raised to power y	pow(2,3) = 8

Example:

```
import math

print(math.sqrt(16))
print(math.pow(2, 5))
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

4. Rounding Functions

Function	Description	Example
`math.ceil(x)`	Smallest integer ≥ x	`ceil(4.2) = 5`
`math.floor(x)`	Largest integer ≤ x	`floor(4.8) = 4`
`math.trunc(x)`	Removes decimal part	`trunc(4.9) = 4`

Example:

```python
import math

print(math.ceil(4.2))
print(math.floor(4.8))
print(math.trunc(4.9))
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

5. Absolute and Factorial Functions

Function	Description	Example
`math.fabs(x)`	Absolute value of x	`fabs(-7) = 7`
`math.factorial(x)`	Factorial of x	`factorial(5) = 120`

Example:

```python
import math

print(math.fabs(-10))
print(math.factorial(5))
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

8. Important Utility Functions

Function	Description
`math.gcd(a, b)`	Greatest common divisor
`math.lcm(a, b)`	Least common multiple (Python 3.9+)
`math.isqrt(x)`	Integer square root

Example:

```python
import math

print(math.gcd(12, 18))
print(math.lcm(12, 18))
print(math.isqrt(20))
```


ATIL UNIVERSITY NAVI MUMBAI

Summary Table of Common Functions

Category	Functions
Constants	pi, e
Powers/Roots	sqrt(), pow()
Rounding	ceil(), floor(), trunc()
Absolute	fabs()
Factorial	factorial()
Logarithmic	log(), log10()
Trigonometric	sin(), cos(), tan()
Utility	gcd(), lcm(), isqrt()

D Y PATIL UNIVERSITY
NAVI MUMBAI

Common Built-in String Methods

Method	Description	Example
lower()	Converts to lowercase	"HELLO".lower() → hello
upper()	Converts to uppercase	"hello".upper() → HELLO
title()	Title case	"hello world".title()
capitalize()	Capitalizes first letter	"python".capitalize()
strip()	Removes spaces	" hi ".strip()
replace(a,b)	Replaces substring	"cat".replace("c","b")
split()	Splits into list	"a,b,c".split(",")
join()	Joins list into string	",".join(["a","b"])
find()	Finds index	"hello".find("e")
count()	Counts occurrences	"banana".count("a")

D Y PATIL UNIVERSITY
NAVI MUMBAI

String Constants in string Module:

Constant	Description	Example Output
string.ascii_lowercase	Lowercase letters	abcdefghijklmnopqrstuvwxyz
string.ascii_uppercase	Uppercase letters	ABCDEFGHIJKLMNOPQRSTUVWXYZ
string.ascii_letters	All letters	a-z and A-Z
string.digits	Numeric characters	0123456789
string.hexdigits	Hexadecimal digits	0-9, a-f, A-F
string.octdigits	Octal digits	01234567
string.punctuation	Special characters	!”#$%&’()*+,-./:;<=>?@[]^_`{
string.whitespace	Space, tab, newline	' \t\n\r\x0b\x0c'

D Y PATIL UNIVERSITY
NAVI MUMBAI

String Operators:

Operator	Description	Example
+	Concatenation	"Hello" + "World"
*	Repetition	"Hi" * 3
in	Membership	"a" in "apple"

D Y PATIL UNIVERSITY
NAVI MUMBAI

Feature	String Methods	String Module
Purpose	Modify strings	Provide constants
Import required	No	Yes (import string)
Example	"abc".upper()	string.ascii_letters

D Y PATIL UNIVERSITY
NAVI MUMBAI

Python Random Module

Introduction

The **Random module** in Python is used to generate:

- Random numbers
- Random selections
- Random sequences

It is commonly used in:

- Simulations
- Games
- Testing
- Password generation
- Sampling data

To use the module:

```
import random
```


D Y PATIL UNIVERSITY
NAVI MUMBAI

Commonly Used Random Functions

Function	Description		Example
random()	Returns random float between 0 and 1	random()	0.56
randint(a, b)	Returns random integer between a and b (inclusive)	randint(1,10)	7
randrange(start, stop, step)	Random number from a range	randrange(1,10)	5
choice(seq)	Selects random element from sequence	choice(list)	"red"
choices(seq, k=n)	Selects multiple random elements	choices(list,k=3)	[2,4,2]
shuffle(seq)	Shuffles a list randomly	shuffle(list)	[3,1,5,2]
sample(seq, k)	Selects unique random elements	sample(list,3)	[1,4,5]
uniform(a, b)	Random float between a and b	uniform(1,5)	3.67

D Y PATIL UNIVERSITY
NAVI MUMBAI

Thank You

D Y PATIL
UNIVERSITY
NAVI MUMBAI